

AWS RDS

GCP Cloud SQL

AWS RDS vs GCP Cloud SQL

托管关系型数据库全面对比：引擎 · 部署模式 · 存储 · HA/SLA · 特色能力

对标关系：先厘清层次

本 PPT 聚焦 RDS vs Cloud SQL（托管传统引擎），Aurora/AlloyDB 仅附注

对比维度	AWS RDS	GCP Cloud SQL
托管传统引擎	RDS ← 本对比主角	Cloud SQL ← 本对比主角
云原生高性能	Aurora （MySQL/PG 兼容）	AlloyDB （PostgreSQL 兼容）
全球分布式	Aurora Global / Aurora DSQL	Spanner

说明：

- RDS 与 Cloud SQL 都是「全托管传统数据库引擎」，是最直接的同类对标。
- Aurora / AlloyDB 是各自的云原生高性能引擎，架构是另一档，本次不展开。
- 全球分布式场景（Spanner / Aurora Global）不属于同类，仅列出对应关系。

1. 支持的数据库引擎

RDS 引擎覆盖更广，独有 Oracle / Db2 / MariaDB

对比维度	AWS RDS	GCP Cloud SQL
引擎数量	6 种	3 种
MySQL / PostgreSQL	✓ / ✓	✓ / ✓
SQL Server	✓	✓
MariaDB	✓	✗
Oracle	✓（全托管）	✗（走 Oracle@Google Cloud）
IBM Db2	✓（全托管）	✗（无对标托管服务）

2. GCP 上如何跑 Oracle / Db2 ?

Cloud SQL 不支持，需走其它路径

Oracle → 有官方方案

Oracle Database@Google Cloud

Google × Oracle 合作：把 OCI Exadata 硬件放进 GCP 数据中心，低延迟直连 + IAM 集成。

支持的 OCI 服务：

- Exadata Database Service
- Autonomous AI Database Service
- Base Database Service
- Exascale Infrastructure / GoldenGate

本质：不是 Cloud SQL 托管，是在 GCP 里用原生 OCI Oracle 服务。

Db2 → 无对标托管

GCP 没有等价的全托管 Db2 服务

（Cloud SQL 不支持，也无 Oracle 那样的合作方案）

客户只能：

- ① Compute Engine 上自建 Db2（非托管）
- ② IBM Cloud 的 Db2 托管（跨云）
- ③ 迁移到 PostgreSQL / 其它引擎

→ 这是 GCP 相对 AWS 的真实短板：

AWS RDS 有全托管 Db2。

3. 版本模型： Cloud SQL 分 Edition

Cloud SQL 两档打包能力； RDS 靠实例类 + 选项自由组合

对比维度	AWS RDS	GCP Cloud SQL
模型	无 edition 概念，靠实例类 + 选项组合	Enterprise / Enterprise Plus 两档
可用性 SLA	统一（见 SLA 页）	Ent 99.95% / Ent Plus 99.99%
Data cache（本地 SSD 读缓存）	对应 Optimized Reads（本地 NVMe）	仅 Enterprise Plus，读性能最高 4x
优化写	Optimized Writes（NVMe 16KiB 原子写）	仅 Ent Plus，写吞吐 3x / 延迟 -98%
PITR 日志保留	0-35 天	Ent 最多 7 天 / Ent Plus 最多 35 天
计划维护停机	维护窗口内（可 Blue/Green 近零）	Ent 数分钟 / Ent Plus 亚秒级

来源： docs.cloud.google.com/sql/docs/mysql/editions-intro（官方 edition 对比表）

4. RDS 三种部署模式（含本地 NVMe binlog）

Multi-AZ DB cluster 用本地 NVMe 存事务日志 /binlog 降写延迟

① Single-AZ instance

架构：单实例，单 AZ

用途：开发 / 测试，无高可用

② Multi-AZ DB instance

架构：1 主 + 1 备用 (standby, 不可读)，2 AZ，同步复制

用途：高可用，故障自动切换，备用不分担读

③ Multi-AZ DB cluster

架构：1 写 + 2 可读 reader，跨 3 AZ，半同步复制

用途：HA + 读扩展 + 更低写延迟

关键：③ Multi-AZ DB cluster 「binlog/ 事务日志写本地 NVMe」

- 强制使用带 d 后缀的本地 NVMe 实例类 (db.m6gd/m6id/r6gd/r5d/x2iedn...)
- RDS 官方特性页原文: "Uses local storage for transactional logs to reduce jitter"
- binlog 走本地 NVMe(快) + 半同步复制到 2 个 AZ reader → 写延迟低于 Multi-AZ DB instance

5. 高可用 & SLA

HA 基础档打平； Cloud SQL Enterprise Plus 高一档； RDS 单实例有兜底

对比维度	AWS RDS	GCP Cloud SQL
HA / 多可用区 SLA	99.95% （ Multi-AZ 实例 / 集群）	99.95% （ Ent ） / 99.99% （ Ent Plus ）
单实例 SLA	99.5% （ Single-DB Instance 有兜底）	单区 / 共享核实例不在 SLA 覆盖内
HA 机制	同步复制到备用副本，自动故障转移	跨 zone 主备，故障自动切换
读扩展 HA	Multi-AZ DB cluster （ 2 可读 reader ）	Read pool （ Ent Plus ， 1-20 节点）

6. 存储上限 & 只读副本

RDS 存储可调性更强；副本模型两家不同

对比维度	AWS RDS	GCP Cloud SQL
最大存储	64 TiB ； Oracle/SQLServer 附加卷 256 TiB	64 TB （专用核） / 3 TB （共享核）
持久块存储	gp3 / io2 Block Express / io1	SSD / HDD
最高 IOPS	io2 Block Express 256,000 IOPS	随规格自动走，无独立 provisioned IOPS
本地 NVMe	d 系实例类 (≤7.6TB) + Optimized Reads	Enterprise Plus data cache （本地 SSD ）
只读副本上限	每主实例最多 15 个（硬上限）	副本 = 实例，受项目配额 ≤ 1000/ 项目

7. 备份 / PITR & 特色能力

各自亮点

AWS RDS 特色

- 自动备份 0-35 天 + PITR 秒级恢复点
- Blue/Green 部署 (近零停机升级 / 切换)
- RDS Proxy (连接池、故障转移加速)
- Performance Insights (性能分析)
- RDS Custom (可访问 OS/DB, Oracle/SQLServer)
- 引擎最全 (6 种, 含 Oracle/Db2)
- Optimized Reads / Optimized Writes (本地 NVMe)

GCP Cloud SQL 特色

- PITR: Ent 7 天 / Ent Plus 35 天
- near-zero downtime 计划维护 (Ent Plus 亚秒)
- Managed Connection Pooling (Ent Plus)
- Query Insights + AI 辅助排障 (Ent Plus)
- IAM 数据库认证深度集成 + Auth Proxy
- Read pools (Ent Plus, 1-20 节点统一读端点)
- Data cache 读 4x / 优化写 3x (Ent Plus)

云原生篇： Aurora vs AlloyDB

存算分离的云原生数据库对标（RDS/Cloud SQL 之上的一档）

Aurora

AlloyDB

8. 云原生架构：存算分离

两者都是「基于 LOG 的共享存储 + 读写分离」（墨天轮解析印证）

对比维度	AWS Aurora	GCP AlloyDB
兼容引擎	MySQL + PostgreSQL（双引擎）	仅 PostgreSQL
存储模型	cluster volume：数据 6 副本跨 3 AZ	disaggregated 存算解耦，跨多 AZ
存储分层	分布式共享存储卷（SSD）	log storage + LPS(日志处理) + 块存储 三层
binlog/ 日志	binlog 存在 cluster volume 内	WAL 转日志写入 log storage 层
只读副本	最多 15 个 Aurora Replica，共享存储	主节点 + read pool，共享 LOG 存储
临时文件	本地存储放排序 / 建索引临时文件	计算节点本地资源

9. HTAP / 列式引擎 / AI （AlloyDB 差异化）

AlloyDB 内置列存内存 + WAL Apply 优化，主打 HTAP

对比维度	AWS Aurora	GCP AlloyDB
HTAP / 列式	本体偏 OLTP ，分析走 Redshift/zero-ETL	内置 columnar engine （列存内存），原生 HTAP
列存装载方式	—	手工装载：SELECT google_columnar_engine_add('表')
只读节点优化	共享存储读	WAL Apply 优化：只读节点仅重演主实例 CACHE 数据，其余直接读数据文件
AI 能力	pgvector 扩展 + Bedrock/SageMaker 集成	AlloyDB AI：向量搜索 +ML 调用内置进引擎
实现方式	AWS 自研引擎	通过 PG Extension 实现，随 PG 社区版演进

10. AlloyDB 命名 & 性能宣称 (分层标注)

厂商 benchmark 互不承认，命名无官方逐字解释

「Alloy」= 合金：为什么这么取名？

Alloy(合金)=把多种金属熔合成性能更强的新材料。AlloyDB 寓意把「开源 PostgreSQL」熔合「Google 自研 scale-out 计算 / 存储 + AI/ML」= 更强的新产品。

△ Google 无「官方逐字」命名解释文档；此为基于官方定位 (blog: "combines the best of Google's... with full PostgreSQL compatibility") 的合理推断。

性能宣称 (△ 均为厂商自家 benchmark，互不承认，选型须自测)

- AlloyDB(GCP 自述)：比原生 PG 事务 4x、分析 100x；比 Amazon 同类事务 2x
- Aurora(AWS 自述)：比标准 MySQL 5x、PG 3x
- 第三方 (墨天轮)：AlloyDB 大型统计 SQL 实测提升几十倍，但非严格对照；AlloyDB 未像 HeatWave 那样堆硬件砸性能

引擎

要 MySQL 云原生只能 Aurora；AlloyDB 只做 PostgreSQL

架构

都是 LOG 共享存储 + 读写分离；Aurora 6 副本 / 3AZ，AlloyDB 存算解耦三层

差异化

AlloyDB 内置列存 HTAP + AlloyDB AI (向量 / ML 内置引擎) 更激进

全球分布式篇： Aurora Global / DSQL vs Spanner

跨区域分布式数据库对标（云原生之上的一档）

Aurora Global / DSQL

Cloud Spanner

11. 全球分布式：三方对标

真正对标 Spanner 的是 Aurora DSQL （active-active），不是 Aurora Global （单主）

维度	AWS Aurora Global / DSQL	GCP Cloud Spanner
定位	Global= 跨区灾备 + 就近读 DSQL= 全球分布式 serverless SQL	全球分布式 SQL （多模型）
写模式	Global= 单主 (1 区写) DSQL=active-active 多区可写	多区，Paxos leader 写
兼容	MySQL/PG （Global） PG 兼容 （DSQL）	专有 + PostgreSQL 接口
复制	Global= 存储级异步 <1s DSQL= 同步强一致	同步 Paxos ，多数派提交
RPO/RTO	Global: RPO<1s, RTO<1min DSQL: RPO=0 自动恢复	RPO=0 ， Paxos 多数派容错

12. 一致性机制 & SLA

Spanner 靠 TrueTime ， DSQL 靠 OCC ； 多区 SLA 都是 99.999%

对比维度	AWS Aurora DSQL/Global	GCP Cloud Spanner
一致性模型	DSQL：强一致 +snapshot isolation(OCC) Global：主区强一致 / 从区 <1s 滞后	external consistency （最强） + TrueTime
时钟 / 并发	DSQL：QP 互不通信，OCC 乐观并发	TrueTime(GPS+ 原子钟) + MVCC + Paxos
多区 SLA	DSQL 多区 99.999% / 单区 99.99%	Multi/Dual-Regional 99.999% / Regional 99.99%
多模型	纯关系型 (PG 兼容)	关系 +Graph+ 全文 + 向量 一库多模
成熟度	DSQL 2024 re:Invent 新出，生态较新	2017 GA ，成熟

13. Spanner 命名 & 强一致的代价

全球强一致牺牲了什么？CAP 与延迟 trade-off

「Spanner」命名来头

寓意「span 跨越」——一个数据库横跨全球多个数据中心 / 区域（global database that spans the world）。

△ Google 无官方命名解释；此为社区共识（Wikipedia/Reddit），非官方逐字定义。

CAP：Spanner 技术上是 CP 系统（牺牲 A，非 CA）

Eric Brewer(CAP 定理提出者 /Google VP) 官方博客原话："during some partitions, Spanner chooses C and forfeits A. It is technically a CP system."

但靠 Google 私有全球光纤网，网络分区极罕见（网络仅占 <10% 故障）→ 实测 >5 个 9(99.999%)，用户「当它是 CA」用。

Latency：牺牲的是「写延迟」（读靠 MVCC 基本不受影响）

- ① TrueTime commit-wait：写事务要等时钟不确定性区间 ϵ 过去才提交（保证 external consistency）→ 写多等 ϵ (毫秒级)
- ② 跨区写要 Paxos 多数派同步 → 写延迟受跨区网络 RTT/ 光速制约（跨洲几十 ~ 上百 ms）
- ③ 读：MVCC 多版本，snapshot read 不阻塞写、可本地副本完成 → 读延迟低。核心 trade-off = 写延迟 ↑ 换 全球强一致 + 高可用

总结：关键差异一览

选型时优先看这几点

引擎覆盖

RDS 完胜（6 vs 3，独有 Oracle/Db2/MariaDB）；GCP Oracle 靠 Oracle@GCP，Db2 无对标

HA / SLA

HA 基础档打平 99.95%；Cloud SQL Ent Plus 99.99% 高一档；RDS 单实例 99.5% 兜底

存储可调性

RDS 更强：io2 Block Express 256K IOPS、256TiB 附加卷、独立 provisioned IOPS

部署模式

RDS 三档；Multi-AZ DB cluster 用本地 NVMe 存 binlog/ 事务日志降写延迟（强制 d 系）

版本模型

Cloud SQL 用 Enterprise/Ent Plus 分档打包能力；RDS 靠实例类 + 选项自由组合

本地 NVMe

两家都重度用本地 NVMe/SSD 加速，但用途不同：RDS 临时对象 / 原子写 vs Cloud SQL 读缓存

全球分布式

对标 Spanner 的是 Aurora DSQL(active-active)；Spanner=CP 系统靠 TrueTime，写延迟换全球强一致